

# CIC – 209: DATA STRUCTURES

## Unit – I Overview of Data Structure and Abstract Data Types

### Data Type

- A data type is characterized by:
  - *A set of values that the variable may assume*
  - *A data representation, which is common to all these values, and*
  - *A set of operations, which can be applied uniformly to all these values*

Data Type = A Set of Permitted Values + A Set of Allowed Operations

OR

A Set of Permitted Values + A Set of Allowed Operations = Data Type

## Data Structure (1)

- **Definition 1:** A data structure is an aggregation of atomic and composite data into a set with defined relationships.
- **Definition 2:** A data structure is representation of the logical relationship existing between individual elements of data.
- **Definition 3:** A data structure is a way of organizing all data items that considers not only the elements stored but also their relationship to each other.
- In the above definitions structure means a set of rules that holds the data together. In other words, if we take a combination of data and fit them into a structure such that we can define its relating rules, we have made a data structure.
- The relationships are usually specified by specifying operations on one or more instances.
- Data structures can be nested.

## Data Structure (1)

**Program = Algorithm + Data Structure**

**OR**

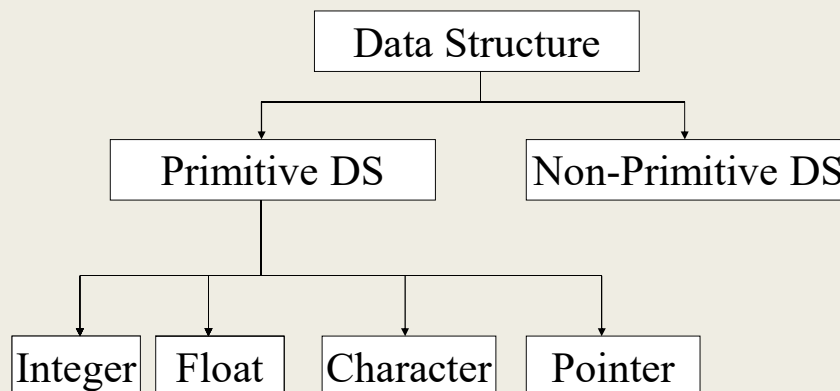
**Algorithm + Data Structure = Program**

- Algorithm is a set of instructions (or steps) written to carry out certain tasks & the data structure is the way of organizing the data with their logical relationship retained.
- To develop a program of an algorithm, we should select an appropriate data structure for that algorithm.

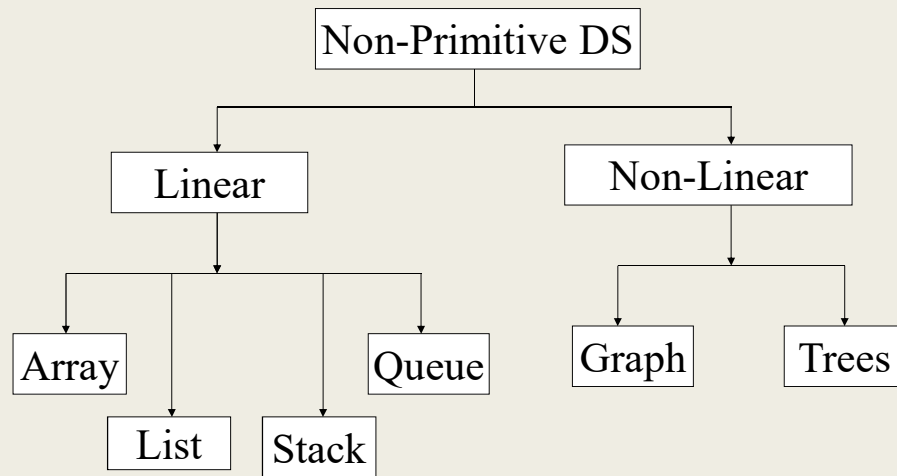
## Classification of Data Structures

- **Primitive / Non-primitive**
  - *Basic Data Structures available / Derived from Primitive Data Structures*
- **Homogeneous / Heterogeneous**
  - *Elements are of the same type / Different types*
- **Static / Dynamic**
  - *memory is allocated at the time of compilation / run-time*
- **Linear / Non-linear**
  - *Linear if its elements form a sequence or a linear list e.g. array, stacks, queues and linked lists organize data in linear order.*
  - *Non linear if its elements form a hierarchical classification where, data items appear at various levels e.g. trees and graphs represents hierarchical relationship between individual data elements.*

## Classification of Data Structures (1)



## Classification of Data Structures (2)



## Primitive Data Structure

- These are basic structures and directly operated upon by the machine instructions.
- In general, there are different representation on different computers.
- Integer, Floating-point number, Character constants, String constants, Pointers etc, fall in this category.

## Non-Primitive Data Structure

- These are more sophisticated data structures.
- These are derived from the primitive data structures.
- The non-primitive data structures emphasize on structuring of a group of homogeneous (same type) or heterogeneous (different type) data items.
- Lists, Stack, Queue, Tree, Graph, etc. are the examples of non-primitive data structures.
- The design of an efficient data structure must take operations to be performed on the data structure.

## Non-Primitive Data Structure

- **Common Operations**
  - *Creation*
  - *Traversing (or Visiting)*
  - *Searching*
  - *Inserting*
  - *Deleting*
- **More Operations**
  - *Selection*
  - *Updating*
  - *Sorting*
  - *Merging*

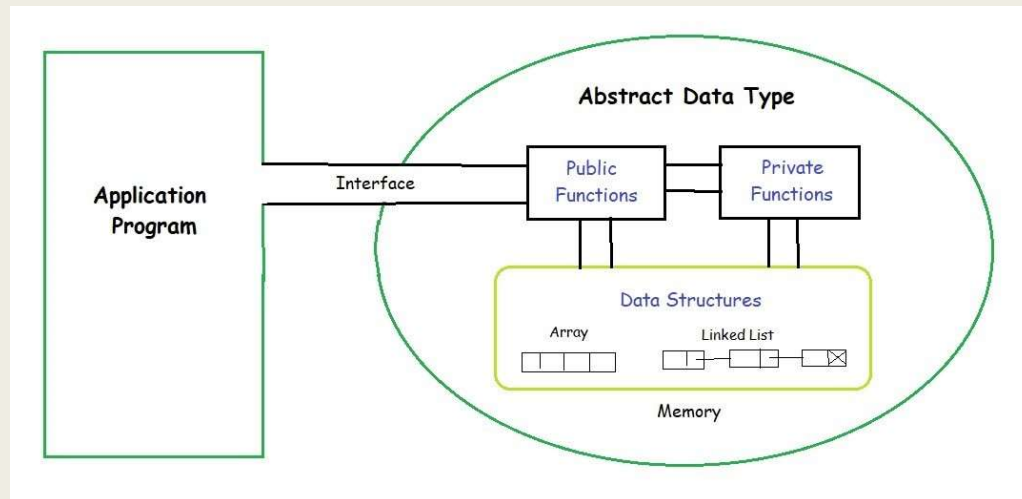
## Abstract Data Type (ADT) (1)

- The abstract notion of a data structure is defined in terms of the operations that are planned to be performed on the data.
- Considering both the organization of data and the expected operations on the data, leads to the notion of an abstract data type.
- With an ADT, users are not concerned with **how** the task is done but rather with **what** it can do.
- In other words, the ADT consists of a set of definitions that allow programmers to use the functions while hiding the implementation.
- **This generalization of operations with unspecified implementations is known as abstraction.**
- We abstract the essence of the process and leave the implementation details hidden.
- **Definition 1:** An abstract data type is a theoretical construct that consists of data as well as the operations to be performed on the data while hiding implementation.
- **Definition 2:** An abstract data type can be thought of as a mathematical model for data types with a collection of operations defined on that model.
- **Definition 3:** An abstract data type is an encapsulation of the data and the operations on the data which are hidden from the user.

## Abstract Data Type (ADT) (2)

- The two properties – generalization and encapsulation – apply equally well to ADTs.
  - ADTs are generalizations of primitive data types, just as procedures are generalizations of primitive operations.
  - The ADT encapsulates data type in the sense that the definition of the type and all operations on that type can be localized to one section of program.
- An abstract data type (ADT) implements a set of algorithms generically so that they can be applied to any data type or construct.
- The beauty of an ADT implementation is that the algorithms can handle any data type whether it is a simple integer or a complex record.
- In simple words, an abstract data type is:
  - 1. Declaration of data
  - 2. Declaration of operations
  - 3. Encapsulation of data and operations
- An ADT specifies:
  - Data stored
  - Operations on the data
  - Error conditions associated with operations

## Model for an Abstract Data Type



## Abstract Data Type Model (1)

### ■ ADT Operations

- Data are entered, accessed, modified, and deleted through the external interface drawn as a passageway partially in and partially out of the ADT.
- Only the public functions are accessible through this interface.
- For each ADT operation there is an algorithm that performs its specific task.
- Only the operation name and its parameters are available to the application, and they provide the only interface to the ADT.

### ■ ADT Data Structure

- Because the abstract data type must hide the implementation from the user, however, all data about the structure must be maintained inside the ADT.
- Just encapsulating the structure in an ADT is not sufficient. Consequently, it must hide the implementation from the user while being able to store different data.

## Abstract Data Type Model (2)

### ■ ADT Implementations

#### – Array Implementations

- In an array, the sequentiality of a list is maintained by the order structure of elements in the array (indexes).
- Although searching an array for an individual element can be very efficient, addition and deletion of elements are complex and inefficient processes.
- For this reason arrays are infrequently used, especially when the list changes frequently.
- In addition, array implementations of nonlinear lists can become excessively large, especially when there are several successors for each element.

#### – Linked List Implementations

- A linked list is an ordered collection of data in which each element contains the location of the next element or elements.
- A linked list can be used to create linear and non-linear structures.
- In linear linked lists, each element has only zero or one successor.
- In non-linear linked lists, each element can have zero, one, or more successors.
- The major advantage of the linked list over the array is that data are easily inserted and deleted. It is not necessary to shift elements of a linked list to make room for a new element or to delete an element.
- On the other hand, because the elements are no longer physically sequenced, we are limited to sequential searches.

## ADT for Stock Trade

- The data stored are buy/sell orders

- The operations supported are

- *order buy (stock, shares)*
- *order sell(stock, shares )*
- *void cancel(order)*

- Error conditions:

- *Buy/sell a nonexistent stock*
- *Cancel a nonexistent order*



## Why ADT?

- Modularity
  - *divide program into small functions*
  - *easy to debug and maintain*
  - *easy to modify*
  - *group work*
- Reuse
  - *do some operations only once*
- Easy to change the implementation
  - *transparent to the program*